UAlg FCT
UNIVERSIDADE DO ALGARVE
FACULDADE DE CIÊNCIAS E TECNOLOGIA

The design pattern Template Method

Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides, *Design Patterns – Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1995, AKA GoF

Bruce Eckel, *Thinking in Patterns with Java*, cf. http://www.mindview.net/Books/TIPatterns/

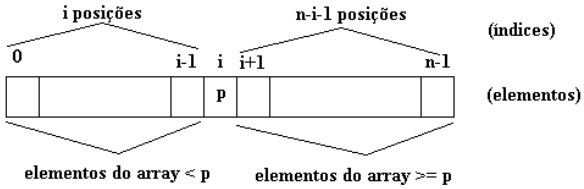
1

A case study

- We want to implement the several versions of quicksort reusing code as much as possible

2

Re: Is this a common *type* of problem ?

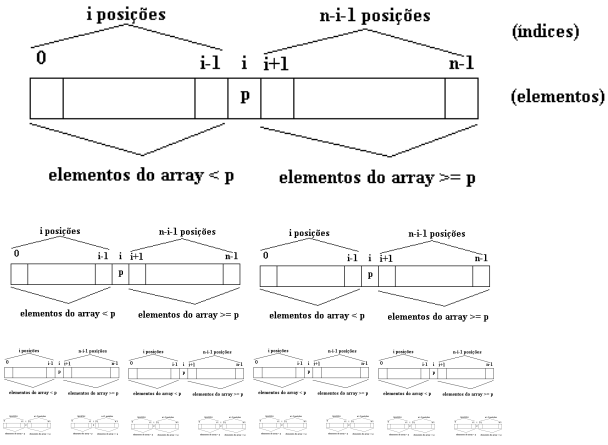


Quicksort basics:

1. Let an element be the pivot
2. Rearrange the remaining elements in two groups:
   - The group with elements (whose keys are) less than the pivot (key)
   - The group with elements (whose keys are) great or equal to the pivot (key)
3. Apply the above method to both groups

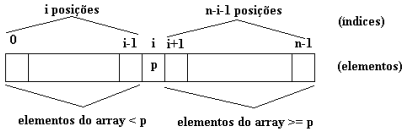
3

Re: Is this a common *type* of problem ?



4

Re: Is this a common *type* of problem ?



Quicksort basics:

1. Let an element be the pivot
2. Rearrange the remaining elements in two groups:
   - The group with elements (whose keys are) less than the pivot (key)
   - The group with elements (w… pivot (key)
3. Apply the above method to…

```
void quicksort(int left, int right) {
    if (right <= left) return;

    int i = partition(left, right);

    quicksort(left, i-1);
    quicksort(i+1, right);
}
```


5

Re: How should our client code looks like?

Command line

```
> Java Main 8 3 1 <Enter>
1
3
8
```

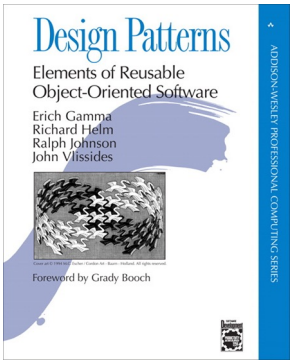
```
public class Main {
   void main() {
      Integer[] a = new Integer[args.length];
      for (int i=0; i<args.length; i++)
                  a[i] = Integer.parseInt(args[i]);
      ProbQuickSort pqs = new ProbQuickSort();
      pqs.quicksort(a);
      for(int i=0; i< a.length; i++ )
                  System.out.println (a[i]);
   }
}
```


6

Previously on OOP: Pattern

"Pattern is a **named** and well-known problem/solution pair that can be applied in **new contexts**, with advice on how to apply it in novel situations and discussion of its **trade-offs**, **implementations**, **variations**, and so forth." (Craig Larman, *Applying UML and Patterns*, 1995)

From a pattern catalogue: Template Method

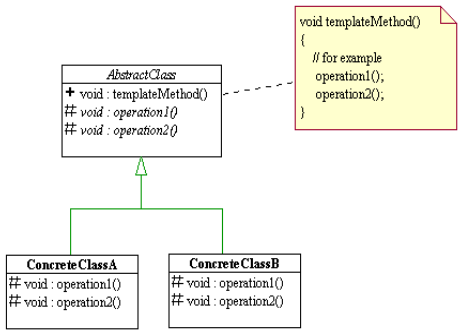


- Intent
  - To define a skeleton algorithm by deferring some steps to subclasses
  - To allow the subclasses to redefine certain steps

Template Method is used:

- To implement the invariant parts of an algorithm once and let the subclasses to implement the behavior that can vary.
- When common behavior among subclasses should be factored and localized in a common class to avoid code duplication.
- To control subclasses extensions. That is, to define a specific points where extensions are permited.

Template Method - Structure



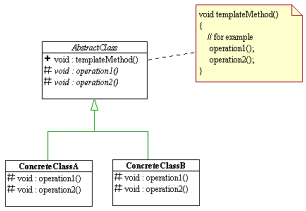
A Quicksort class as the Template Method abstract class

```
@SuppressWarnings("unchecked")

abstract class Quicksort {
   private Comparable[] data;

    public final void quicksort(Comparable[] d) {
      data = d;
      quicksort(0, data.length-1);
   }

    /** quicksort method as the template method */
    private final void quicksort(int left, int right) {
        if (right <= left) return;
        int i = partition(left, right);
        quicksort(left, i-1);
        quicksort(i+1, right);
   }
```



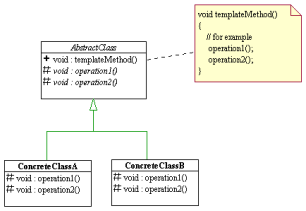
Methods for subclasses to use

```
/* Call this in a subclass for swaping elements i and j */
protected final void exchange(int i, int j) {
    Comparable tmp = data[i];
    data[i] = data[j];
    data[j] = tmp;
}

/** Use this in subclasses to compare elements
     x.compareTo(y) answer 0 if x ==y; -1 if x < y; 1 se x>y
 */
protected final boolean less(int i, int j) {
    return (data[i].compareTo(data[j]) < 0);
}
```

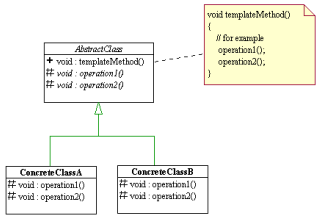
ProbQuickSort as a concrete class

```
class ProbQuickSort extends Quicksort {
   @Override
   protected int partition(int left, int right) {
      int last=left;
      int pp = left + (int) (Math.random() * (right-left)); // pivot position
      exchange(left, pp);
      for(int i=left+1; i <= right; i++)
          if (less(i, left)) exchange(i, ++last);
      exchange(left, last);
      return last;
   }
}
```



Example of another concrete class

```
class ClassicalQuickSort extends Quicksort {
  @Override
  protected int partition(int left, int right) {
     int l2r=left-1;              // left to right index
     int pp=right-1;              // pivot position
     int r2l=pp;                  // right to left index
     for(;;) {
           do ++l2r;
           while(less(l2r, pp)) ;
           do if(--r2l < 0) break;
           while(less(pp, r2l));
           if(l2r >= r2l) break;
           exchange(l2r, r2l);
     }
     exchange(pp, l2r);
     return l2r;
   }
}
```



Recalling client code

```
public class Main {
    void main() {
        Integer[] a = new Integer[args.length];
        for (int i=0; i<args.length; i++)
                        a[i] = Integer.parseInt(args[i]);
        QuickSort qs = new ClassicalQuickSort();
        qs.quicksort(a);
        for(int i=0; i< a.length; i++ )
                        System.out.println (a[i]);
    }
}
```


Recalling client code

```
public class Main {
    void main() {
        Integer[] a = new Integer[args.length];
        for (int i=0; i<args.length; i++)
                        a[i] = Integer.parseInt(args[i]);
        QuickSort qs = new ProbQuickSort();
        qs.quicksort(a);
        for(int i=0; i< a.length; i++ )
                        System.out.println (a[i]);
    }
}
```

Template Method – brief discussion

- Template Method is a fundamental design pattern for code reuse.
- It's particularly important in class libraries, as it is intent for factoring out common behavior in the class hierarchy.
- It leads to an inverted control structure know as the *Hollywood Principle* ("Don't call us, we'll call you").
- This pattern is so fundamental that it can be found in almost every abstract class.

17

How-to check list

- Examine the algorithm, and decide which steps are standard and which steps are peculiar
- Define a new abstract base class to host the "don't call us, we'll call you" framework.
- Move the shell of the algorithm (now called the "template method") and the definition of all standard steps to the new base class.
- Define a placeholder or "hook" method in the base class for each step that requires many different implementations. This method can host a default implementation – or it can be defined as abstract
- Invoke the hook method(s) from the template method.
- Each of the existing classes declares an "is-a" relationship to the new abstract base class.
- Remove from the existing classes all the implementation details that have been moved to the base class.
- The only details that will remain in the existing classes will be the implementation details peculiar to each derived class.

18

Take home today

- A case study: the multiple versions of quicksort
- The design pattern Template Method